Case

Elite Treinos

Introdução

Você vai desenvolver um sistema web chamado **Elite Treinos**.

A ideia é simples e muito real: uma plataforma onde **personais** gerenciam seus **alunos** e atribuem **treinos** (baseados em modelos já existentes), enquanto o **aluno** faz login e **apenas visualiza** os treinos que recebeu.

O objetivo desse case é avaliar como você estrutura um projeto **fullstack** (API + Front separado) do zero até a entrega final, lidando com:

- autenticação e autorização por perfil (RBAC)
- CRUDs com regras de acesso e segurança real no back-end
- modelagem de dados e relacionamento (Personal → Alunos → Treinos)
- API bem desenhada (Laravel) + UI organizada (React)
- documentação de API clara (Swagger/OpenAPI)
- qualidade de código (estrutura, legibilidade, manutenção e segurança básica)

Escopo técnico

Back-end (API)

- PHP 8+
- Laravel (preferencialmente)
- API REST (JSON)
- Seed dos treinos modelo (A/B/C/D)
- Validações de entrada + respostas padronizadas de erro
- Documentação da API via Swagger/OpenAPI
- Banco de dados MySQL ou PostgreSQL

Front-end (Web)

- React 18+
- React Router
- Consumo de API (Axios/fetch)
- Estado (livre)
- UI simples, limpa e utilizável

Perfis de usuário e regras

1) Superadmin

- Faz login
- **Gerencia Personais (CRUD)**
- **Visualiza os alunos de cada Personal** (somente leitura)

2) Personal

- Faz login
- **Gerencia seus Alunos (CRUD)**
- Cria o **login do aluno** (email e senha) ao cadastrar um novo aluno
- Para cada aluno, pode **atribuir até 2 treinos**
- Só pode atribuir treinos baseados nos **modelos pré-cadastrados** (A/B/C/D)

3) Aluno

- Faz login com **email e senha próprios**
- **Apenas visualiza** os treinos atribuídos para ele (somente leitura)

Regras de Treinos

- Existem **4 treinos modelo** já cadastrados no banco: **A, B, C e D**
- O personal pode atribuir **no máximo 2 treinos por aluno**
- O aluno não cadastra nem edita nada: apenas visualiza

Modelo mínimo **(telas que devem existir)**

Autenticação (React)

- Tela de Login
- Redirecionamento pós-login conforme perfil

Superadmin

- Lista de Personais
- Criar/Editar/Excluir Personal
- Detalhe do Personal: **listar alunos daquele personal** (somente leitura)

Personal

- Lista de Alunos
- Criar/Editar/Excluir Aluno (inclui criação do login: email + senha)

- Detalhe do Aluno:
    - visualizar treinos atribuídos
    - atribuir treino (selecionar A/B/C/D)
    - respeitar regra do limite de 2

Aluno

- Tela “Meus Treinos”
    - lista dos treinos atribuídos
    - visualizar detalhes do treino (descrição + exercícios)

Exemplos de treinos **(devem vir pré-cadastrados no banco)**

Treino A — Full Body (Base)

Objetivo: adaptação geral e base de força
Exercícios:

1. Agachamento livre — 3x10
2. Supino reto — 3x10
3. Remada curvada — 3x10
4. Desenvolvimento militar — 3x12
5. Prancha — 3x30–45s

Treino B — Inferiores (Pernas/Glúteos)

Objetivo: foco em pernas e glúteos
Exercícios:

1. Leg press — 4x12
2. Afundo (passada) — 3x10 (cada perna)
3. Mesa flexora — 3x12
4. Cadeira extensora — 3x12
5. Elevação pélvica — 4x10

Treino C — Superiores (Peito/Costas/Ombros)

Objetivo: hipertrofia de tronco
Exercícios:

1. Supino inclinado — 4x10
2. Puxada na barra (pulldown) — 4x10
3. Remada baixa — 3x12
4. Elevação lateral — 3x15
5. Rosca direta — 3x12

Treino D — Condicionamento + Core

Objetivo: condicionamento e core
Exercícios:

1. HIIT na esteira/bike — 10–15 min (30s forte / 60s leve)
2. Burpee — 3x10
3. Abdominal infra — 3x15
4. Prancha lateral — 3x30s (cada lado)
5. Alongamento final — 5 min

Dados (o que precisa ser armazenado)

Usuários (login)

O sistema precisa armazenar, para **todos os tipos de usuário** (superadmin, personal e aluno):

- nome
- e-mail (único)
- senha (armazenada de forma segura)
- nível/perfil de acesso (superadmin, personal, aluno)

Personais

Para cada **personal**, o sistema precisa armazenar:

- vínculo com o usuário de login do personal
- informações básicas de identificação (ex.: telefone, CREF — opcional)

Alunos

Para cada **aluno**, o sistema precisa armazenar:

- vínculo com o usuário de login do aluno (e-mail e senha próprios)
- vínculo com o **personal responsável** (um aluno pertence a um personal)
- informações básicas do aluno (ex.: data de nascimento, observações — opcionais)

Treinos modelo (A, B, C e D)

O sistema precisa ter cadastrados previamente **4 treinos de exemplo**, cada um contendo:

- código/identificador do treino (A, B, C ou D)
- nome do treino
- objetivo
- lista de exercícios do treino, com:
    - ordem do exercício
    - nome do exercício
    - séries
    - repetições/tempo
    - observações (opcional)

Treinos atribuídos ao aluno

O sistema precisa registrar, para cada aluno:

- quais treinos modelo foram atribuídos a ele (referenciando A/B/C/D)
- qual personal atribuiu
- quando foi atribuído
- regra: **no máximo 2 treinos por aluno**
- recomendação operacional: impedir atribuir o mesmo treino duas vezes para o mesmo aluno

---

Documentação da API (Swagger/OpenAPI)

Você deve entregar a documentação dos endpoints da API usando **Swagger/OpenAPI**, contendo:

- todos os endpoints implementados
- payloads de request/response
- exemplos de erro (401/403/422/404)
- definição do esquema de autenticação (Bearer token/cookie, etc.)
- como testar via Swagger UI

O que entregar:

- uma URL local acessível do Swagger UI (ex.: `/api/docs` ou `/docs`)
- e o arquivo `openapi.json`/`openapi.yaml` versionado no repositório

Requisitos mínimos da API

Autenticação

- login
- logout
- identificar usuário logado e seu perfil

Superadmin

- cadastrar/editar/remover/listar personais
- visualizar os alunos vinculados a um personal (somente leitura)

Personal

- cadastrar/editar/remover/listar seus alunos
- criar o login do aluno ao cadastrá-lo
- atribuir treinos modelo a um aluno (respeitando o limite de 2)
- listar treinos já atribuídos para um aluno

Aluno

- listar seus treinos atribuídos
- visualizar detalhes do treino (incluindo exercícios)

Entregáveis

Você deverá subir o projeto em um **repositório público no GitHub** e compartilhar o link, contendo:

- `backend/` (Laravel API completo)
- `frontend/` (React completo)
- `README.md` com:
  - como rodar o backend (instalar dependências, configurar env, rodar migrations/seeds)
  - como rodar o frontend
  - credenciais de teste (um superadmin, um personal, um aluno)
  - decisões técnicas (auth, estrutura, libs)
  - como acessar a documentação Swagger/OpenAPI
- (opcional) collection do Postman/Insomnia

Entrega e apresentação

- **Prazo:** até **domingo, 08 de março, às 23h59**
- Envie para **io@eliteformacoes.com.br** o **link do repositório público** no GitHub
- **Assunto:** “Case - Elite Treinos Fullstack”

Na apresentação, você deverá:

- demonstrar login com os 3 perfis
- mostrar as telas principais funcionando
- mostrar rapidamente o Swagger (documentação dos endpoints)
- explicar arquitetura e decisões (auth/autorização/modelagem)
- explicar como garantiu as regras (ex.: “personal só vê seus alunos” e “máx 2 treinos por aluno”)